Entity States in Entity Framework Core

Back to: ASP.NET Core Web API Tutorials

Understanding Entity States in Entity Framework Core (EF Core)

Now we will understand one of the most **important EF Core concepts** that directly impacts how changes in your entities are tracked, detected, and saved to the database, the **Entity States** and **Change Tracking System.** We will be working with the **same example** that we created in our previous article.

What are Entity States in Entity Framework Core?

In **Entity Framework Core (EF Core)**, every entity object (like Product, Order, Customer, Payment, etc.) that is being tracked by the **DbContext** is assigned a **state**. This **state** tells EF Core what action it should perform on that entity when we call the SaveChanges() method. So, Entity states help EF Core understand whether:

- A record should be **Inserted** (Added),
- **Updated** (Modified),
- **Deleted** (Deleted),
- **Ignored** (Unchanged).

In Simple Terms, EF Core maintains a kind of "memory" of all entities it's tracking. Whenever we load or modify entities, EF Core keeps track of their current values and compares them to their original values (from the database).

When we work with a DbContext, we are working **in-memory**, nothing is saved to the database until SaveChanges() is called. Before EF Core can generate SQL commands for saving data, it needs to know *what* happened to each entity. That's where **Entity States** come in, they guide EF Core's SQL generation process.

When EF Core Tracks an Entity

An entity becomes **tracked** by EF Core when:

- It is retrieved from the database via DbSet queries (Find, FirstOrDefault, etc.).
- It is added to the context using .Add(), .Update(), or .Attach().

Every tracked entity immediately receives a **state** which EF Core manages internally through the **Change Tracker**.

The Five Entity States in Entity Framework Core

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

The entity state is represented by an enum called EntityState in EF Core with the following signature.

```
namespace Microsoft.EntityFrameworkCore
{
    public enum EntityState
    {
        Detached,
        Unchanged,
        Deleted,
        Modified,
        Added
    }
}
```

As you can see, there are five states of an entity, and an entity is always in any one of the above states. Let us proceed and understand these states in detail:

Added

When an entity is marked as **Added**, it means the object is new and does not yet exist in the database. EF Core will generate an **INSERT** SQL command for it when SaveChanges() is called. For example:

```
{
    Name = "Bluetooth Mouse",
    Price = 1200,
    Stock = 10
};

_context.Products.Add(product);
```

Here:

- EF Core assigns this object the state **Added**.
- The object exists only in memory until you call SaveChanges().
- Once saved, EF Core inserts it into the Products table and changes its state to **Unchanged**.

Unchanged

When an entity is fetched from the database and no modifications are made to it, EF Core marks it as **Unchanged**. This means the entity's data is already synchronized with the database. For example**:**

- **var product = _context.Products.FirstOrDefault(p => p.Id == 1);**

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Here:

- EF Core knows that the product object represents a record in the database.
- Since no modifications are made yet, the state is **Unchanged**.
- EF Core will **not** generate any SQL command for this entity during SaveChanges().

Modified

When you retrieve an entity from the database, change one or more of its property values, and then call SaveChanges(), EF Core marks it as **Modified**. It will then generate an **UPDATE** statement for the changed columns. For example**:**

```
var product = _context.Products.FirstOrDefault(p => p.Id == 1);
product.Price = 1500;  // Changed property
product.Stock = 20;    // Another modification
_context.SaveChanges(); // EF Core issues an UPDATE command
```

Here,

- EF Core internally detects that property values have changed.
- It marks the entity's state as **Modified**.
- It executes an SQL UPDATE query to persist changes.

After successfully updating the record, the entity's state resets to **Unchanged**.

Deleted

When an entity is marked as **Deleted**, EF Core knows that this record should be removed from the database. On calling SaveChanges(), EF Core generates a **DELETE** SQL command. For example:

```
var product = _context.Products.Find(1);
_context.Products.Remove(product);
_context.SaveChanges();
```

Here,

- Before SaveChanges() → State is **Deleted**.
- After saving → EF Core executes DELETE FROM Products WHERE Id = 1.
- The object remains in memory, but the record is removed from the database.

Detached

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

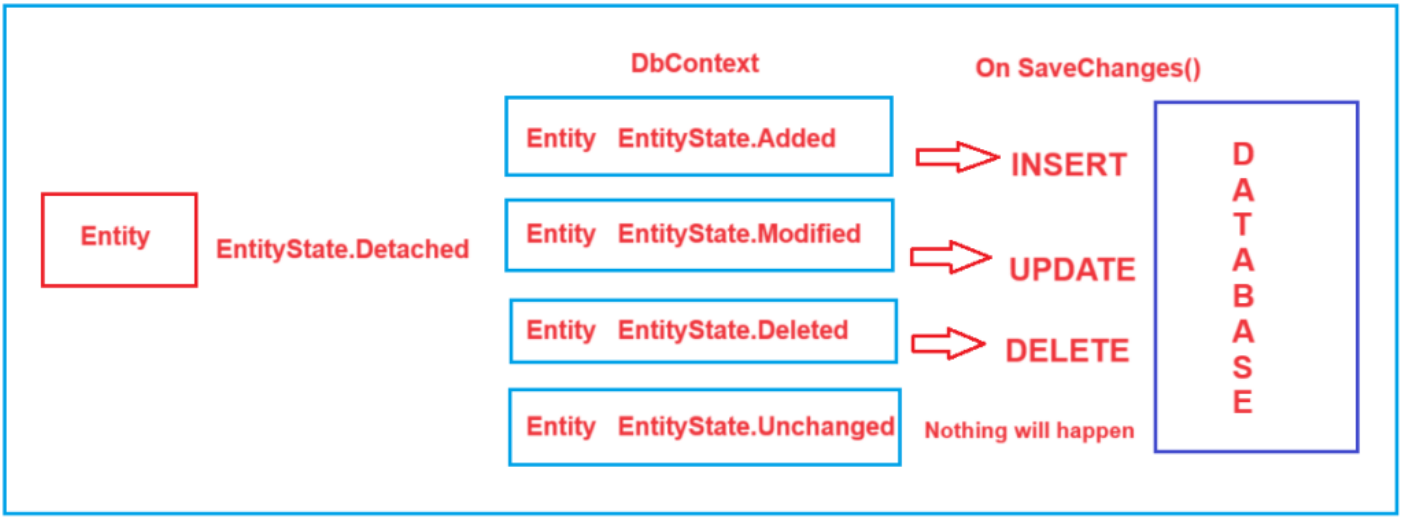
When an entity is **not being tracked** by any DbContext instance, its state is **Detached**. EF Core doesn't know anything about it, and no SQL operation will be generated for this object unless you attach it manually. For example:

```
var product = new Product { Id = 10, Name = "Monitor" };
var entry = _context.Entry(product);
Console.WriteLine(entry.State);  // Detached
```

Entity Lifecycle in Entity Framework Core (EF Core)

When working with **Entity Framework Core**, every entity instance (a C# class mapped to a database table) goes through a **lifecycle** within the DbContext. This lifecycle determines how EF Core **tracks**, **detects**, and **persists** changes to the database.

An **entity's lifecycle** is managed through **Entity States**, which define how EF Core interprets the current status of that object, whether it needs to be inserted, updated, deleted, or ignored when SaveChanges() is called. For a better understanding, please have a look at the following image.



EF Core internally maintains a **Change Tracker**, which monitors the state of each entity being tracked by the DbContext. The change tracker detects modifications, additions, or deletions and updates the entity's EntityState accordingly.

- When you **query** data, EF Core automatically starts tracking those entities (in Unchanged state).
- When you **modify** a property value, EF Core automatically marks it as Modified.
- When you **add** or **remove** an entity manually using DbSet methods, EF Core explicitly changes its state to Added or Deleted.

Important Note: The **only automatic state change** handled by EF Core is from **Unchanged** → **Modified** when you edit a tracked entity's property. All other transitions (Added, Deleted, Detached) must be performed manually using context methods such as Add(), Update(), or Remove().

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Example: Demonstrating All Entity States

Let's modify our **ProductController** to demonstrate how EF Core assigns entity states in real-time. Pease read the inline comments for a better clarity.

```
using Microsoft.AspNetCore.Mvc;
using ProductManagementAPI.Data;
using ProductManagementAPI.Models;

namespace ProductManagementAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductController : ControllerBase
    {
        private readonly AppDbContext _context;

        public ProductController(AppDbContext context)
        {
            _context = context;
        }

        // ------------------ 1. ADDED STATE ------------------
        [HttpPost("AddProduct")]
        public IActionResult AddProduct([FromBody] Product product)
        {
            /*
             * Step 1: When a new entity (product) is created and added using
.Add(),
             *         EF Core marks it as 'Added' in the ChangeTracker.
             * Step 2: When SaveChanges() is called, EF Core executes an INSERT
statement.
             * Step 3: After successful save, EF Core automatically changes its
state to 'Unchanged'
             *         because now it matches the database record.
             */

            _context.Products.Add(product); // State => Added
            var beforeSave = _context.Entry(product).State.ToString(); // "Added"

            _context.SaveChanges(); // Executes INSERT

            var afterSave = _context.Entry(product).State.ToString(); //
"Unchanged"

            return Ok(new
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
            {
                Message = "New Product Added Successfully",
                StateBeforeSave = beforeSave,
                StateAfterSave = afterSave
            });
        }

        // ------------------- 2. UNCHANGED STATE -------------------
        [HttpGet("GetProduct/{id}")]
        public IActionResult GetProduct(int id)
        {
            /*
             * Step 1: When you fetch an entity using Find() or FirstOrDefault(),
             *         EF Core starts tracking it immediately.
             * Step 2: Since the entity is freshly loaded and no property has
changed yet,
             *         its state is 'Unchanged'.
             * Step 3: If you later modify it, the state will automatically
become 'Modified'.
             */

            var product = _context.Products.FirstOrDefault(p => p.Id == id);

            if (product == null)
                return NotFound("Product not found.");

            var stateAfterFetch = _context.Entry(product).State.ToString(); //
"Unchanged"

            return Ok(new
            {
                product,
                StateAfterFetching = stateAfterFetch
            });
        }

        // ------------------- 3. MODIFIED STATE -------------------
        [HttpPut("UpdateProduct/{id}")]
        public IActionResult UpdateProduct(int id, [FromBody] Product
updatedProduct)
        {
            /*
             * Step 1: Retrieve an existing product → EF Core sets its state as
'Unchanged'.
             * Step 2: When you modify one or more properties (like Price or
Stock),
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
             *         EF Core automatically changes the entity's state to
'Modified'.
             * Step 3: When SaveChanges() is called, EF Core executes an UPDATE
statement.
             * Step 4: After saving successfully, EF Core resets its state back
to 'Unchanged'.
             */

            var product = _context.Products.Find(id);
            if (product == null)
                return NotFound("Product not found.");

            var stateAfterFetch = _context.Entry(product).State.ToString(); //
"Unchanged"

            // Modify one or more properties
            product.Price = updatedProduct.Price;
            product.Stock = updatedProduct.Stock;

            var stateAfterModify = _context.Entry(product).State.ToString(); //
"Modified"

            _context.SaveChanges(); // Executes UPDATE command

            var stateAfterSave = _context.Entry(product).State.ToString(); //
"Unchanged" again

            return Ok(new
            {
                Message = "Product Updated Successfully",
                StateAfterFetching = stateAfterFetch,
                StateAfterModification = stateAfterModify,
                StateAfterSaveChanges = stateAfterSave
            });
        }

        // ------------------- 4. DELETED STATE -------------------
        [HttpDelete("DeleteProduct/{id}")]
        public IActionResult DeleteProduct(int id)
        {
            /*
             * Step 1: When you fetch an entity, EF Core marks it as 'Unchanged'.
             * Step 2: When you call .Remove(), EF Core marks it as 'Deleted'.
             * Step 3: When SaveChanges() is called, EF Core executes a DELETE
SQL command.
             * Step 4: After deletion, EF Core changes its state to 'Detached'
             *         because it no longer exists in the database.
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
             */

            var product = _context.Products.Find(id);
            if (product == null)
                return NotFound("Product not found.");

            var stateAfterFetch = _context.Entry(product).State.ToString(); //
"Unchanged"

            _context.Products.Remove(product); // Marks entity as Deleted
            var stateAfterRemove = _context.Entry(product).State.ToString(); //
"Deleted"

            _context.SaveChanges(); // Executes DELETE command
            var stateAfterSave = _context.Entry(product).State.ToString(); //
"Detached" (no longer tracked)

            return Ok(new
            {
                Message = "Product Deleted Successfully",
                StateAfterFetching = stateAfterFetch,
                StateAfterRemove = stateAfterRemove,
                StateAfterSaveChanges = stateAfterSave
            });
        }

        // ------------------- 5. DETACHED STATE -------------------
        [HttpGet("DetachedExample")]
        public IActionResult DetachedExample()
        {
            /*
             * Step 1: When you create a new entity instance manually and do not
attach or add it to the DbContext,
             *         EF Core has no tracking information for it → its state is
'Detached'.
             * Step 2: Detached means EF Core won't insert, update, or delete it
unless you explicitly attach it.
             */

            var product = new Product
            {
                Id = 100,
                Name = "Portable Charger",
                Price = 2500,
                Stock = 15
            };
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
            var initialState = _context.Entry(product).State.ToString(); //
"Detached"

            // Now attach it manually (simulate connecting it to context)
            _context.Attach(product);
            var stateAfterAttach = _context.Entry(product).State.ToString(); //
"Unchanged"

            // Modify property to see state transition
            product.Price = 3000;
            var stateAfterModification =
_context.Entry(product).State.ToString(); // "Modified"

            _context.SaveChanges(); // Executes UPDATE command if entity exists
            var stateAfterSave = _context.Entry(product).State.ToString(); //
"Unchanged" again

            return Ok(new
            {
                Message = "Detached Entity Demonstration Complete",
                InitialState = initialState,
                AfterAttach = stateAfterAttach,
                AfterModification = stateAfterModification,
                AfterSaveChanges = stateAfterSave
            });
        }

        // ------------------- 6. SHOW ALL TRACKED ENTITIES -------------------
        [HttpGet("ShowTrackedEntities")]
        public IActionResult ShowTrackedEntities()
        {
            /*
             * Displays all entities currently tracked by the DbContext's
ChangeTracker,
             * along with their entity type and current state.
             * This helps visualize which entities EF Core is keeping in memory
and how it perceives them.
             */

            var trackedEntities = _context.ChangeTracker.Entries()
                .Select(e => new
                {
                    EntityName = e.Entity.GetType().Name,
                    CurrentState = e.State.ToString()
                }).ToList();

            return Ok(trackedEntities);
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
        }
    }
}
```

What Does Attach() Do?

The Attach() method in Entity Framework Core is used to **Start Tracking** an entity **without marking it for any database changes**. In simple words: Attach() tells EF Core: “This entity already exists in the database. Please start tracking it, but don’t try to insert or update it unless I make changes manually.”

So, when you call: **_context.Attach(product);** EF Core **does not generate any SQL immediately**. It simply starts tracking the entity in the **Change Tracker** and marks its state as **Unchanged**.

What Exactly Is Change Tracking?

When we work with EF Core, we are not directly writing SQL. Instead, EF Core acts as an **Object Relational Mapper (ORM)**, its **Change Tracker** keeps an in-memory record of:

- All entities it is currently “watching”
- Each entity’s **current values** and **original (snapshot) values**
- The **Entity State** (Added / Modified / Deleted / Unchanged / Detached)

The Change Tracker lives inside DbContext instance and acts like a “transaction notebook.” Every entity you query or attach becomes an “entry” inside that notebook.

Where Does EF Core Store Tracked Entities?

Inside every DbContext, there’s a property called: **context.ChangeTracker.** Internally, this object maintains a collection of **EntityEntry** instances. Each EntityEntry represents one tracked entity and stores metadata about it.

For example, when we query data: **var product = context.Products.FirstOrDefault(p => p.Id == 1);** EF Core performs the following behind the scenes:

1. **Creates the Entity Instance** (Product) and populates its properties.
2. **Registers the Entity** with the context’s ChangeTracker.
3. **Takes a snapshot** of its original property values (stored internally as key–value pairs).
4. Marks its **EntityState** = Unchanged.

So inside memory, EF Core keeps an internal dictionary roughly like:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
ChangeTracker.Entries:
└── [Product: Id=1]
    ├── CurrentValues: { Id=1, Name="Mouse", Price=1200, Stock=20 }
    ├── OriginalValues: { Id=1, Name="Mouse", Price=1200, Stock=20 }
    └── Entity State: Unchanged
```

Whenever we change a property, EF Core compares the CurrentValues and OriginalValues to determine whether something has changed.

How Does EF Core Detect Changes?

When an entity is first loaded or attached, EF Core stores a copy of all property values in a “snapshot” (usually an instance of EntityEntry class). Then, whenever we call SaveChanges():

1. EF Core loops through all tracked entities in **ChangeTracker.Entries().**
2. For each property, it compares the **Current Value** (from the live object) with the **Original Value** (from the snapshot).
3. If any property has changed, EF Core:
    - Marks the entity’s EntityState = Modified
    - Records which columns actually changed (important for generating minimal SQL)

This comparison happens in memory; no database call is made.

How EF Core Uses the Change Tracker During SaveChanges()

When you call: **context.SaveChanges();** EF Core performs the following sequence:

1. **Inspect all tracked entities** from ChangeTracker.Entries().
2. For each entity:
    - Checks its **EntityState**.
    - If it’s Added → Prepare an INSERT statement.
    - If Modified → Compare OriginalValues vs CurrentValues and prepare an UPDATE for changed columns only.
    - If Deleted → Prepare a DELETE statement.
    - If Unchanged → Ignore it.
3. All generated SQL commands are batched into a single **transaction**.
4. Once committed successfully:
    - The OriginalValues snapshot is replaced with the latest values.
    - EntityState for all affected entities is reset to Unchanged.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Entity States and Change Tracking form the **foundation of EF Core's data management**. They allow you to work with .NET objects naturally while EF Core efficiently handles database operations behind the scenes. By understanding how states change and how SaveChanges() translates them into SQL, you gain precise control over your application's database interactions — making your Web API more predictable, efficient, and maintainable.

Registration Open – Mastering Design Patterns, Principles, and Architectures using .NET

New Batch Starts: 11th March, 2026

Session Time: 6:30 AM – 08:00 AM IST

Advance your career with our expert-led, hands-on live training program. Get complete course details, the syllabus, and Zoom credentials for demo sessions via the links below.

View Course Details & Get Demo Credentials

Registration Form

Join Telegram Group

Join WhatsApp Group

Contact: +91 70218 01173 (Call / WhatsApp)

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent | Do Not Sell My Information